Solana 代币架构

SPL Token、账户模型与比特币对比

本文系统整理 Solana 代币系统的核心概念，包括：

SPL Token 标准、Program & Account 模型、Mint / Token Account 结构、
ATA、Token-2022，以及与 Ethereum / Bitcoin 的本质差异。

学习笔记 / 教学讲义版

2026 年 3 月 21 日

目录

1 章节导读 **2**

2 SPL Token 标准 **2**
2.1 什么是 SPL Token 2
2.2 与 Ethereum ERC-20 的根本差异 2

3 程序与账户分离：Solana 的核心设计 **3**
3.1 什么是 Program 3
3.2 什么是 Account 3

4 代币系统中的三类关键对象 **4**
4.1 系统结构图 4
4.2 Mint Account：代币定义账户 4
4.3 Token Account：用户余额账户 5
4.4 一个用户为什么会有多个 Token Account 5

5 Associated Token Account（ATA） **6**
5.1 为什么需要 ATA 6
5.2 ATA 的作用 6
5.3 ATA Program 6

6 Token Program 与 Token-2022 **6**
6.1 标准 Token Program 6
6.2 Token-2022 7

7 Authority：权限控制模型 **7**
7.1 为什么权限设计很重要 7

8 与比特币的对比：为什么说 Solana 代币更原生 **8**
8.1 比特币原生并不支持通用代币模型 8
8.2 外部索引解释 vs 链上原生执行 8

9 从开发者角度重新理解整个系统 **8**
9.1 最小实例：创建一种自己的代币 9

10 初学者最容易混淆的点 **9**
10.1 误区一：Mint Account 就是钱包 9
10.2 误区二：钱包地址本身直接存放所有代币余额 9
10.3 误区三：SPL Token 表示每个代币都有独立程序 9
10.4 误区四：标准化就表示绝对安全 9

11 本章总结 **10**

附录：一句话教学版总结 **10**

1 章节导读

本章的核心问题只有一个：**Solana 上的代币到底是如何存在、被创建、被管理、被转移的？**

在以太坊中，我们通常会把“代币”理解为一份独立部署的 ERC-20 智能合约；而在 Solana 中，代币并不是“每个项目各自部署一套发币逻辑”，而是大多数项目共同使用**官方统一的 Token Program**，再通过创建不同的数据账户来表示不同的币种与用户余额。

因此，理解 Solana 代币系统，关键不在于“这份代币代码长什么样”，而在于理解以下三件事：

1. 什么是 **SPL Token** 标准；
2. 什么是 **Program 与 Account 分离**；
3. 什么是 **Mint Account**、**Token Account**、**Associated Token Account**。

2 SPL Token 标准

2.1 什么是 SPL Token

SPL 是 **Solana Program Library** 的缩写。SPL Token 则是 Solana 生态中最常见、最基础的代币标准。

它的作用类似于以太坊中的 ERC-20：定义了一套标准化的代币行为，例如：

- 创建代币（Create Mint）
- 铸造代币（Mint To）
- 转移代币（Transfer）
- 销毁代币（Burn）
- 冻结与解冻（Freeze / Thaw）
- 授权与委托（Approve / Revoke）

SPL Token 并不意味着“每个代币部署一份自己的程序”。相反，大多数代币都是**调用同一个官方 Token Program**，只是链上对应的 **Mint Account** 与 **Token Account** 不同。

2.2 与 Ethereum ERC-20 的根本差异

维度	Ethereum (ERC-20)	Solana (SPL Token)
实现方式	每个代币通常是一份独立智能合约	全网共用一个 Token Program
创建代币	部署新的 Solidity 合约	调用 Token Program，创建相关账户

状态存放位置	通常由合约内部状态变量保存	由链上账户存储，Program 负责逻辑
审计成本	每个代币合约需单独审计	标准程序复用，基础逻辑更统一
灵活性	可任意自定义代币逻辑	更标准化，可借助 Extensions 扩展
安全性来源	很大程度依赖开发者代码质量	基础逻辑由经过广泛使用的标准程序提供

一个非常常见的比喻是：

- **Ethereum 模式**：每个人自己造一台印钞机；
- **Solana 模式**：大家共用一台认证过的标准印钞机，但用不同模版印不同的币。

3 程序与账户分离：Solana 的核心设计

3.1 什么是 Program

在 Solana 上，**Program** 是链上的可执行代码。它类似智能合约，但与很多初学者熟悉的以太坊合约不同，Solana 的 Program 更强调：

Program 负责执行业务逻辑；Account 负责存储状态数据。

也就是说，Program 本身通常不直接“内嵌存储所有业务状态”，而是接收外部传入的账户，按照规则读取或修改这些账户中的数据。

Program 是规则执行器，不是所有数据的容器。

对于代币系统来说，**SPL Token Program** 就是负责执行代币逻辑的程序。它定义了：

- 什么样的账户可以作为 Mint；
- 什么样的账户可以作为 Token Account；
- 什么条件下允许铸造；
- 什么条件下允许转账；
- 哪些 Authority 有权进行敏感操作。

3.2 什么是 Account

Account 是 Solana 上最基础的数据存储单元。每个账户通常具有：

- **地址**（Address）：一个唯一公钥；
- **SOL 余额**：用于支付账户租金或维持其存在；

- **Owner**：哪个 Program 有权修改该账户的数据；
- **Data**：程序定义的结构化状态。

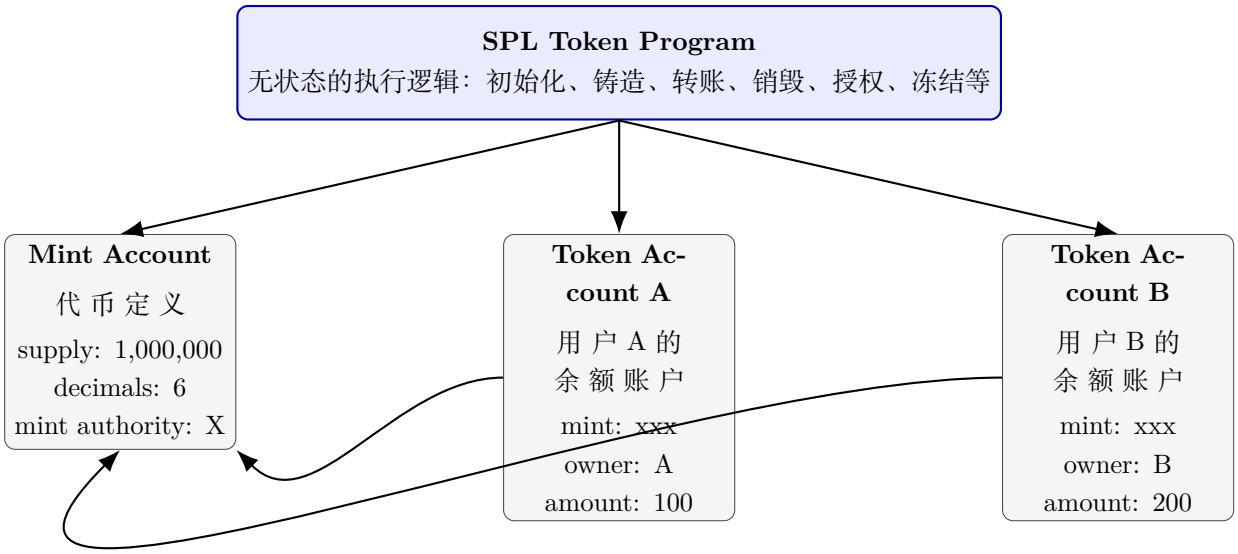
这里的“账户 owner”指的是：**哪个程序可以改这个账户的数据**。这和“这个代币账户属于哪个用户”是两个不同层次的概念。

4 代币系统中的三类关键对象

在 Solana 代币系统中，最关键的不是“代币合约”，而是以下对象之间的关系：

1. **Token Program**：负责执行业务逻辑；
2. **Mint Account**：定义代币本身；
3. **Token Account**：记录某个用户持有某个代币的余额。

4.1 系统结构图



4.2 Mint Account：代币定义账户

Mint Account 可以被理解为：

“某一种代币的全局定义与发行配置中心”

它通常包含以下信息：

- **total supply**：总供应量；

- **decimals**：精度；
- **mint authority**：谁有权继续增发；
- **freeze authority**：谁有权冻结相关账户；
- 其他元数据指针（视实现而定）。

Mint Account **不存储任何具体用户的余额**。它记录的是“这是什么币，以及它的全局发行规则”，而不是“谁有多少”。

4.3 Token Account：用户余额账户

Token Account 是代币系统中的余额载体。它代表：

“某个用户持有某种代币的数量”

一个 Token Account 中通常会包含：

- **mint**：该账户对应的是哪一种代币；
- **owner**：这个余额账户属于哪个钱包；
- **amount**：当前持有多少代币；
- 其他状态字段（委托、冻结状态等）。

因此，Mint Account 与 Token Account 的关系可以概括为：

- **Mint Account**：定义“币种”；
- **Token Account**：记录“某个用户在该币种下的余额”。

4.4 一个用户为什么会有多个 Token Account

因为一个用户通常会持有多种代币。在 Solana 中，不同代币的余额通常分别记录在不同的 Token Account 中。

例如，一个钱包同时持有：

- USDC
- BONK
- 自己发行的 TEST Token

那么这个钱包通常会对应至少三个 Token Account：

- 一个用于存放 USDC；
- 一个用于存放 BONK；
- 一个用于存放 TEST Token。

钱包地址本身不是所有代币余额的直接存放处。在 Solana 上，代币余额通常存放在各自对应的 Token Account 中。

5 Associated Token Account（ATA）

5.1 为什么需要 ATA

理论上，一个用户针对同一种 Mint，可以创建多个 Token Account。但这会给钱包显示、转账逻辑、应用集成带来不必要的复杂度。

因此，Solana 引入了 **Associated Token Account（ATA）** 的概念：

某个钱包地址针对某个 Mint 的“标准默认代币账户”

5.2 ATA 的作用

ATA 的主要价值体现在：

- 钱包与应用可以快速确定“默认收款地址”；
- 发送方更容易知道应将代币发到哪里；
- 代币 UI 展示更统一；
- 开发者不需要为最常见场景自定义复杂的账户策略。

例如，对于“用户 Alice 的钱包地址”与“USDC 这个 Mint”这两个输入，系统可以计算出 Alice 对应 USDC 的标准 ATA 地址。这样当别人给 Alice 转 USDC 时，就可以优先转到这个 ATA。

5.3 ATA Program

与 ATA 相关的账户创建与推导逻辑，由 **Associated Token Account Program** 负责支持。它的核心目的不是重新定义代币逻辑，而是提供一个标准化的“默认代币账户机制”。

6 Token Program 与 Token-2022

6.1 标准 Token Program

标准 SPL Token 的核心思想是：

- 用统一的标准程序处理代币行为；
- 用标准结构定义 Mint 与 Token Account；
- 用标准权限模型控制铸造、冻结、转移等操作。

这种设计带来的最大好处是：

- 工具链成熟；
- 钱包支持广泛；
- 行为一致，易于审计与集成；
- 适合绝大多数普通代币场景。

6.2 Token-2022

标准 Token Program 更强调“统一与稳定”，而 **Token-2022** 则是在标准代币模型基础上的扩展版本。

它允许在标准框架之上加入更多能力，例如：

- Transfer Fee
- Metadata Pointer
- Interest Bearing
- Permanent Delegate
- Confidential Transfer（视支持情况）
- 其他 Extension 机制

可以把 Token-2022 理解成：**在保持整体架构一致的前提下，为代币系统增加更多可选扩展能力**。

7 Authority：权限控制模型

Solana 代币系统之所以安全且标准化，一个很重要的原因在于其权限模型相对明确。

在 Mint Account 与 Token Account 的操作中，常见的权限包括：

- **Mint Authority**：有权增发代币；
- **Freeze Authority**：有权冻结相关 Token Account；
- **Token Account Owner**：有权使用自己的代币账户；
- **Delegate**：被授权在一定额度内代为操作。

7.1 为什么权限设计很重要

即使大家都使用同一套 Token Program，项目本身仍然可能因为权限配置不当而带来风险。例如：

- Mint Authority 没有被撤销，意味着项目方可继续无限增发；
- Freeze Authority 仍存在，意味着项目方理论上可冻结用户代币账户；
- Delegate 机制使用不当，可能导致代币被代扣或异常使用。

标准 Program 并不自动等于“完全无风险”。基础逻辑虽更统一，但具体项目的权限配置仍然决定了很多实际风险。

8 与比特币的对比：为什么说 Solana 代币更原生

8.1 比特币原生并不支持通用代币模型

比特币网络最初主要面向点对点电子现金系统，并不提供像 ERC-20 或 SPL Token 这样成熟的原生代币账户模型。

后来的 Ordinals、BRC-20 等协议，更多是通过：

- 在交易数据中嵌入特定格式的信息；
- 再由外部索引器读取并解释；
- 最终“模拟”出类似代币的效果。

也就是说：

比特币链本身并不会像 Solana Token Program 那样，原生理解“Mint / Token Account / Transfer 语义”。

8.2 外部索引解释 vs 链上原生执行

这就是 Solana 与相关“铭文式代币协议”的重要区别。

比较项	Bitcoin + 外部协议解释	Solana + Token Program
代币语义来源	由外部索引器解释嵌入数据	由链上标准 Program 直接执行
余额追踪	依赖链外理解与同步	由链上账户状态直接表示
一致性	更依赖生态约定	更依赖统一标准程序与账户结构
可组合性	相对受限	与钱包、DEX、程序更易集成

因此我们常说：**Solana 上的代币是原生一等公民**。因为其代币行为并不是通过外部解释“附会”出来的，而是通过链上程序与账户模型直接被定义和执行。

9 从开发者角度重新理解整个系统

如果从开发者视角来总结 Solana 代币系统，可以归纳为以下流程：

1. 链上已经存在一个标准 **Token Program**；
2. 你创建一个 **Mint Account**，定义一种新的代币；
3. 你为用户创建与该 Mint 对应的 **Token Account** 或 **ATA**；
4. Token Program 根据权限与指令修改这些账户中的数据；
5. 最终实现铸造、转账、销毁、授权等操作。

9.1 最小实例：创建一种自己的代币

假设开发者要发行一个名为 `YC Coin` 的代币，那么可能发生如下过程：

1. 创建一个新的 **Mint Account**；
2. 设定其 `decimals = 6`；
3. 指定某个钱包为 **Mint Authority**；
4. 为自己的钱包创建与该 Mint 对应的 **ATA**；
5. 调用 **MintTo**，向自己的 ATA 铸造代币；
6. 当需要转账时，调用 **Transfer**，将代币转到对方该 Mint 的 Token Account。

这里最重要的理解不是“我写了一个新的代币程序”，而是：

- 我复用了**同一个标准 Token Program**；
- 我创建了**属于我这个代币的数据账户**；
- 我通过对这些账户进行操作，实现了代币的完整生命周期。

10 初学者最容易混淆的点

10.1 误区一：Mint Account 就是钱包

错误。Mint Account 不是钱包，也不是用户余额账户。它是“某个代币的全局定义账户”。

10.2 误区二：钱包地址本身直接存放所有代币余额

错误。钱包地址本身更像身份入口，而各类 SPL 代币余额通常存放在对应的 Token Account / ATA 中。

10.3 误区三：SPL Token 表示每个代币都有独立程序

错误。绝大多数 SPL Token 并不会各自部署一份代币逻辑程序，而是共用官方 Token Program。

10.4 误区四：标准化就表示绝对安全

错误。标准化降低了很多基础逻辑风险，但项目本身的权限管理、扩展逻辑、外围程序仍可能带来问题。

11 本章总结

本章可以浓缩成一句话：

在 Solana 上，代币不是“一个代币一份合约”，而是“同一个 Token Program 管理许多 Mint 与 Token Account”。

请牢牢记住下面这组映射关系：

概念	含义
SPL Token	Solana 的标准代币规范
Token Program	执行代币逻辑的标准程序
Mint Account	定义一种代币的全局属性与发行规则
Token Account	记录某个用户持有某种代币的余额
ATA	某钱包针对某 Mint 的标准默认代币账户
Authority	控制铸造、冻结、转移授权等敏感操作的权限

当你真正开始写代码时，所有操作几乎都可以回到这套框架来理解：

- **Program** 负责规则；
- **Account** 负责状态；
- **Mint** 定义币；
- **Token Account / ATA** 记录余额。

附录：一句话教学版总结

如果要用最简洁的方式向别人解释 Solana 代币架构，可以这样说：

> 在 Ethereum 中，一个代币通常是一份独立 ERC-20 合约；而在 Solana 中，代币通常不是重新部署一套发币逻辑，而是调用统一的 SPL Token Program，创建一个 Mint Account 来定义币种，再用 Token Account 或 ATA 记录每个用户的余额。换句话说，Solana 将“逻辑”与“状态”分离：Program 负责规则，Account 负责数据。